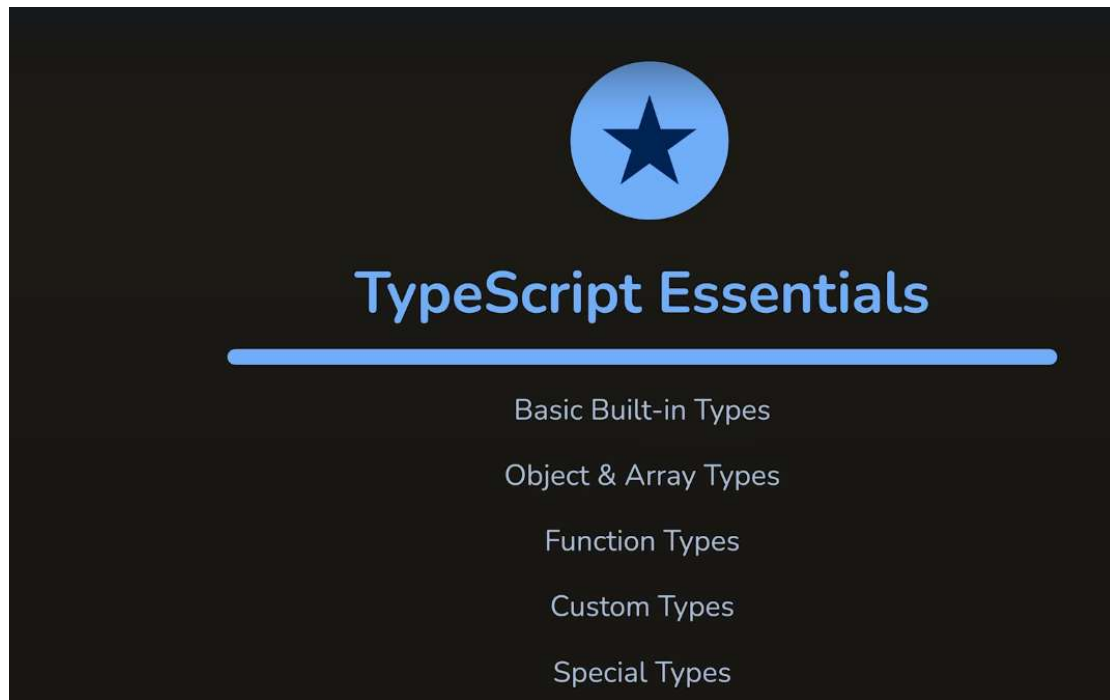
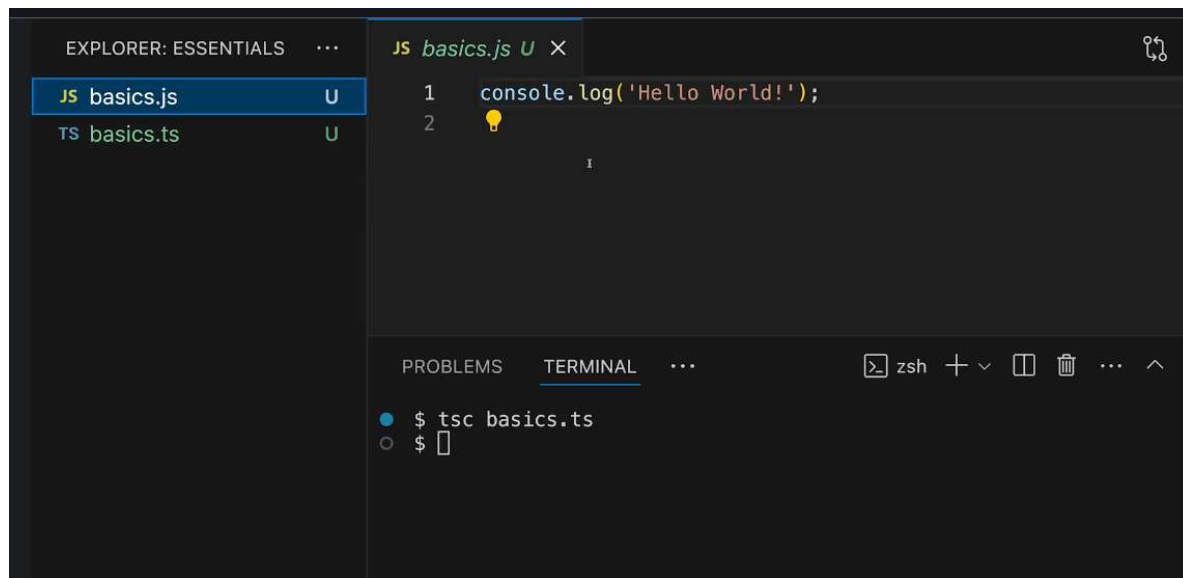


Section 2 - TypeScript Basics And Basic Types

2026年6月1日 9:46



Project Setup



Working with Types and Exploring Built-in Types

```
EXPLORER: ESSENTIALS ... TS basics.ts U X
JS basics.js U
TS basics.ts U

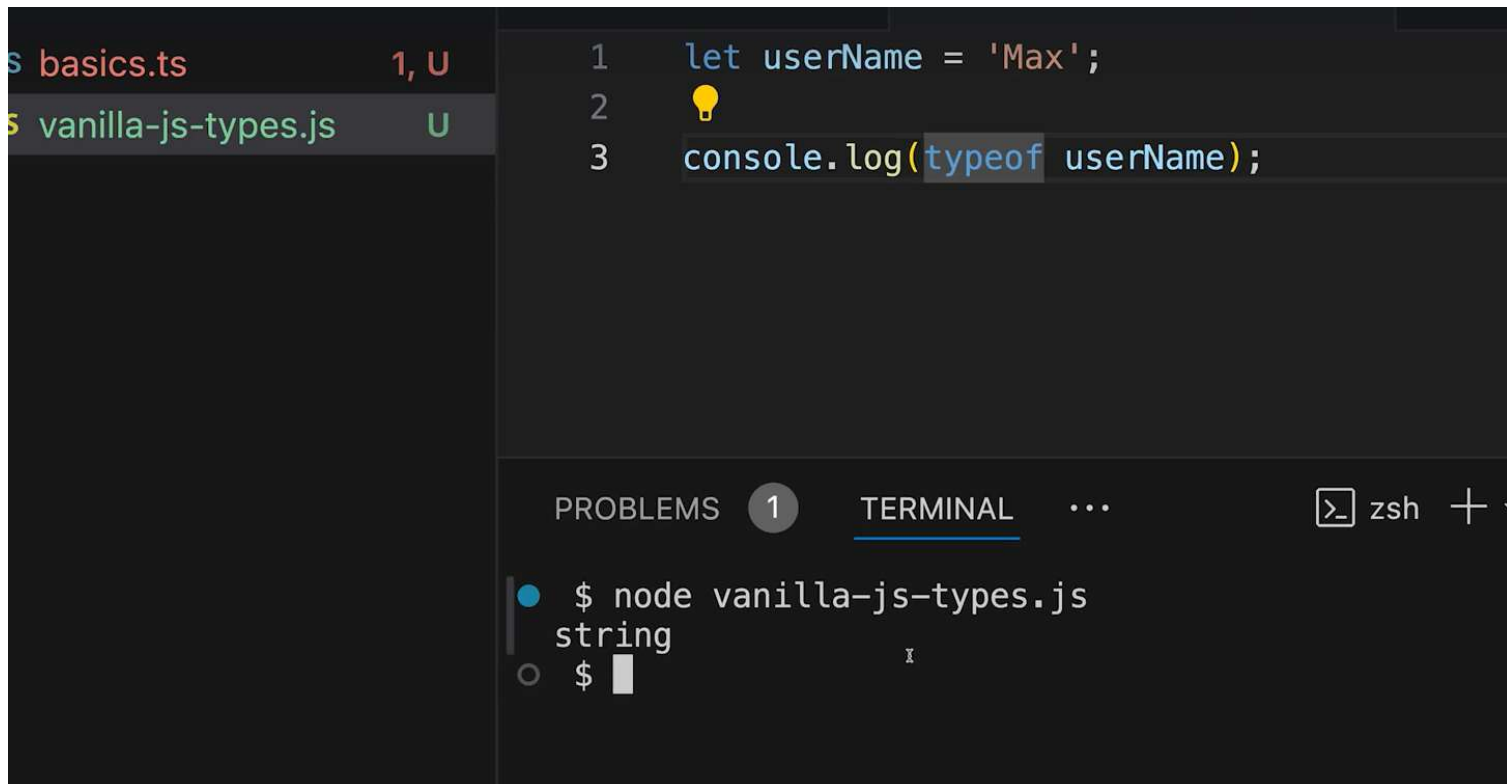
1 let userName;
2
3 // ...
4
5 userName = 'Max';
6
```

The problem with type "userName" is it has any type

```
EXPLORER: ESSENTIALS ... TS basics.ts 1, U JS basics.js U X
JS basics.js U
TS basics.ts 1, U

1 var userName;
2 // ...
3 userName = 'Max';
4
```

Vanilla JavaScript has Types, Too!

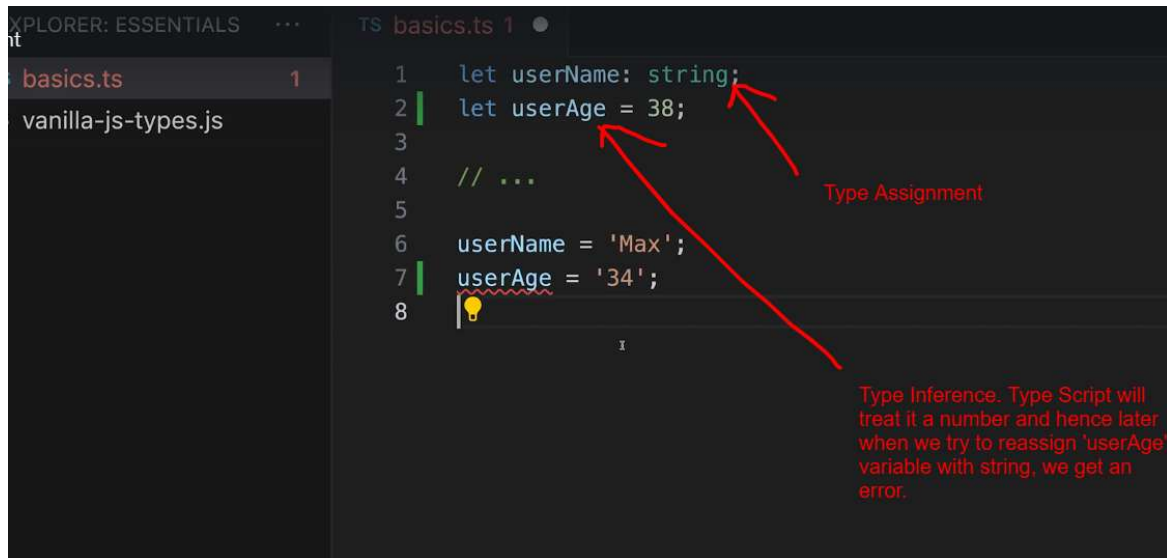


The screenshot shows a VS Code editor with two files: `basics.ts` and `vanilla-js-types.js`. The `basics.ts` file contains the following code:

```
1 let userName = 'Max';
2
3 console.log(typeof userName);
```

The `vanilla-js-types.js` file is empty. The terminal shows the command `$ node vanilla-js-types.js` and the output `string`.

Type Inferences vs Type Assignment



The screenshot shows a VS Code editor with a file named `basics.ts`. The code is as follows:

```
1 let userName: string;
2 let userAge = 38;
3
4 // ...
5
6 userName = 'Max';
7 userAge = '34';
8
```

Red arrows point to the code with the following annotations:

- Type Assignment:** Points to the declaration `let userAge = 38;` on line 2.
- Type Inference:** Points to the reassignment `userAge = '34';` on line 7. A text box explains: "Type Inference. Type Script will treat it a number and hence later when we try to reassign 'userAge' variable with string, we get an error."

Assigning Types to Function Parameters

```
ts basics.ts
js vanilla-js-types.js

1  let userName: string; // number, boolean
2  let userAge = 38;
3
4  // ...
5
6  userName = 'Max';
7  // userAge = '34';
8
9  function add(a: number, b: number) {
10     |   return a + b;
11     |   }

```

```
9  function add(a: number, b = 5) {
10     |   return a + b;
11     |   }
12
13  add(10);

```

```
5
6  userName = 'Max';
7  // userAge = '34';
8  ⚡
9  function add(a: number, b = 5) {
10     |   return a + b;
11     |   }
12
13  add(10);
14  add('10');
15  add(10, 6);
16  add(10, '6');
17
18

```

The "any" type

```
TS basics.ts TS flexible-types.ts U ●
1 let age: any = 36;
2
3 // ...
4
5 age = '37';
6 age = false;
7 age = {};
8 age = [];
```

Understanding Union Types

```
1 let age: string | number = 36;
2
3 // ...
4
```

Array and Types

```
let hobbies = ['Sports', 'Cooking'];
hobbies.push(10);
```

Argument of type 'number' is not assignable to parameter of type 'string'. ts(2345)

[View Problem \(⌘F8\)](#) No quick fixes available

Advanced Array Types

```
TS arrays-objects.ts U ●
1  let hobbies = ['Sports', 'Cooking'];
2
3  // hobbies.push(10);
4
5  let users: (string | number)[];
6
7  users = [1, 'Max'];
8  users = [5, 1];
9  users = ['Max', 'Anna'];
```

A First Glimpse At Generic Types - Alternative Array Type Declaration

```
TS arrays-objects.ts U ●
1  let hobbies = ['Sports', 'Cooking'];
2
3  // hobbies.push(10);
4
5  // let users: (string | number)[];
6  let users: Array<string | number>;
7  |
8  users = [1, 'Max'];
9  users = [5, 1];
10 users = ['Max', 'Anna'];
```

Making Sense of Tuples

```
Making Sense of Tuples TS arrays-objects.ts ●
8  users = [1, 'Max'];
9  users = [5, 1];
10 users = ['Max', 'Anna'];
11
12 let possibleResults: number[]; // [1, -1]
13
14 possibleResults = [1, -1];
15 possibleResults = [5, 10, 12];
```

```

7
8  users = [1, 'Max'];
9  users = [5, 1];
10 users = ['Max', 'Anna'];
11
12 let possibleResults: [number, number]; // [1, -1]
13
14 possibleResults = [1, -1];
15 possibleResults = [5, 10, 12];

```

Object Types

```

TS arrays-objects.ts
14 possibleResults = [1, -1];
15 // possibleResults = [5, 10,
16
17 let user = {
18   name: 'Max',
19   age: 38
20 };

```

```

TS arrays-objects.ts
14 possibleResults = [1, -1];
15 // possibleResults = [5, 10, 12];
16
17 let user: {
18   name: string;
19   age: number;
20 } = {
21   name: 'Max',
22   age: 38
23 };

```

```

15 // possibleResults = [5, 10, 12];
16
17 let user: {
18   name: string;
19   age: number | string;
20   hobbies: string[];
21   role: {
22     description: string;
23     id: number;
24   }
25 } = {
26   name: 'Max',
27   age: 38
28 };

```

```

24   }
25 } = {
26   name: 'Max',
27   age: 38,
28   hobbies: ['Sports', 'Cooking'],
29   role: {
30     description: 'admin',
31     id: 5
32   }
33 };

```

Flexible Objects with the Record Type

```

35 let val: {} = 'is a value';
36
37 let data: Record<>;
38
39 data = '';

```



but it's a flexible type or a flexible kind of object


```

37 const someObj = {
38   name: 'Max'
39 };
40 let data: Record<>;
41
42 data = '';

```

key type is string. in js we can have number key type.

```

34
35 let val: {} = 'is a value';
36
37 const someObj = {
38   'name': 'Max'
39 };
40 let data: Record<>;
41
42 data = '';

```

```

32
33 };
34
35 let val: {} = 'is a value';
36
37 const someObj = {
38   0: 'Max',
39   1: 1
40 };
41 let data: Record<>;
42
43 data = '';

```

```

35 let val: {} = 'is a value';
36
37 let data: Record<string, number | string>;
38
39 data = {
40   entry1: 1,
41   entry2: 'some string'
42 };

```

Working with Enums

```
choices.ts U ●
1 let userRole = 0; // 0 => Admin, 1 => Guest
2
```

```
TS choices.ts U ●
1 enum Role {
2   Admin,
3   Editor,
4   Guest,
5 }
6
7 let userRole: Role = 0;
8
```

I am not getting an error for my 0 here.

```
TS choices.ts 1, U ●
1 enum Role {
2   Admin,
3   Editor,
4   Guest,
5 }
6
7 let userRole: Role = Role.;
8
```

Admin
Editor
Guest
(enum member) Role.Admin = 0

and then you see my IDE gives me some suggestions here.

```

1  enum Role {
2      Admin,
3      Editor,
4      Guest,
5  }
6
7  let userRole: Role = Role.Admin;
8
9  // ...
10
11 userRole = Role.Guest;
12

```

```

1  enum Role {
2      Admin,
3      Editor,
4      Guest,
5  }

```

PROBLEMS DEBUG CONSOLE TERMINAL ...  zsh

```

$ tsc choices.ts
$

```

If I compile this choices.ts file,

```

1  var Role;
2  (function (Role) {
3      Role[Role["Admin"] = 0] = "Admin";
4      Role[Role["Editor"] = 1] = "Editor";
5      Role[Role["Guest"] = 2] = "Guest";
6  })(Role || (Role = {}));
7  var userRole = Role.Admin;
8  // ...
9  userRole = Role.Guest;
10

```

Being Specific With Literal Types

```
choices.ts U X
enum Role {
  Admin,
  Editor,
  Guest,
}

let userRole: Role = 0;

// ...

userRole = Role.Guest;
```

There also is an alternative to using an enum though,

```
choices.ts U ●
1 // enum Role {
2 //   Admin,
3 //   Editor,
4 //   Guest,
5 // }
6
7 let userRole: 'admin' | 'editor' | 'guest' = 'admin';
8
9 // ...
10
11 userRole = 'guest';
12
13 let possibleResults: [1 | -1, number]; // [1, -1]
14
15 possibleResults = [1, -1];
```

```

choices.ts 1, U
6
7 let userRole: 'admin' | 'editor' | 'guest' = 'admin';
8
9 // ...
10
11 userRole = 'guest';
12
13 let possibleResults: [1 | -1, 1 | -1]; // [1, -1]
14 💡
15 possibleResults = [5, -1];

```

↑ error

Type Aliases & Custom Types

```

TS choices.ts
6
7 let userRole: 'admin' | 'editor' | 'guest' | 'reader' = 'admin';
8
9 // ...
10
11 userRole = 'guest';
12
13 let possibleResults: [1 | -1, 1 | -1]; // [1, -1]
14
15 possibleResults = [1, -1];
16
17 function access(role: 'admin' | 'editor' | 'guest' | 'reader') {
18   // ...
19 }

```

```

4 // Guest,
5 // }
6
7 // type MyNumber = number;
8 type Role = 'admin' | 'editor' | 'guest' | 'reader';
9 type User = {
10   name: string;
11   age: number;
12   role: Role;
13   permissions: string[];
14 };
15
16 let userRole: Role = 'admin';
17

```

Function Return Value Types

```

function add(a: number, b: number): number {
  return a + b;
}

```

```

function add(a: number, b: number): string {
  return a + b;
}

```

Type 'number' is not assignable to type 'string'. ts(2322)

[View Problem \(\F8\)](#) No quick fixes available

For many functions though, you don't need it because typescript can infer it.

The "void" type

```
function add(a: number, b: number) {
    return a + b;
}

function log(message: string): void {
    console.log(message);
}
```

The "never" Type

```
5 function log(message: string) {
6     console.log(message);
7 }
8
9 function logAndThrow(errorMessage: string): never {
10     console.log(errorMessage);
11     throw new Error(errorMessage);
12 }
```

Functions As Types

```
8
9 function logAndThrow(errorMessage: string):
10     console.log(errorMessage);
11     throw new Error(errorMessage);
12 }
13
14 function performJob(cb: Function) {
15     // ...
16     cb();
17 }
```

```

12 }
13
14 const logMsg = (msg: string) => {
15   console.log(msg);
16 };
17
18 function performJob(cb: (m: string) => void) {
19   // ...
20   cb('Job done!');
21 }

```

```

5 type User = {
6   name: string;
7   age: number;
8   greet: () => string;
9 };
10
11 let user: User = {
12   name: 'Max',
13   age: 39,
14   greet() {
15     console.log('Hello there!');
16     return this.name;
17   }
18 }

```

```

40 user.greet();

```

null & undefined - Special Types

```

let a: null | string;
a = 'Hi!';

// ...

a = null;

```

Inferred null & A First Look At Type Narrowing


```

<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>TypeScript!</title>
    <script src="form.js" defer></script>
  </head>
  <body>
    <form>
      <p>
        <label>Your name</label>
        <input type="text" id="user-name" />
      </p>
      <button>Submit</button>
    </form>
  </body>

```

```

TS special-types.ts U  <> index.html U  TS form.ts 2, U ●
1  const inputEl = document.getElementById('user-name');
2
3  console.log(inputEl.value);

```

'inputEl' is possibly 'null'. ts(18047)

const inputEl: HTMLElement | null

[View Problem \(⌘F8\)](#) [No quick fixes available](#)

```

const inputEl = document.getElementById('user-name');
const inputEl: HTMLElement | null
console.log(inputEl.value);

```

```

const inputEl = document.getElementById('user-name');

if (!inputEl) {
  throw new Error('Element not found!');
}

const inputEl: HTMLElement
console.log(inputEl.value);

```

Forced "Not Null" And Optional Chaining

```
const inputEl = document.getElementById('user-name')!;  
// if (!inputEl) {  
//   throw new Error('Element not found!');  
// }  
  
console.log(inputEl.value);
```

And therefore, you should be careful with this operator

By writing !, we are telling typescript that, it can not be null.

```
const inputEl = document.getElementById('user-name');  
  
// if (!inputEl) {  
//   throw new Error('Element not found!');  
// }  
//  
console.log(inputEl?.value);
```

And this operator tells JavaScript,

that this value in front of it, inputEl, could be null,

With the question mark operator,

you tell JavaScript and TypeScript

that you know that this could be the case

and that it should only perform the next step, so to say,

and access the value property here if this is not null.

Type Casting

```
1  const inputEl = document.getElementById('user-name');
2  
3  // if (!inputEl) {
4  //   throw new Error('Element not found!');
5  // }
6
7  console.log(inputEl?.value);
```

```
const inputEl = document.getElementById('user-name') as HTMLInputElement | null;

// if (!inputEl) {
//   throw new Error('Element not found!');
// }
console.log(inputEl?.value);
```

The "unknown" Type

```
function process(val: any) {  
  val.log();  
}
```

you can use this value in any way you want

and you'll never get an error.

I can try to call a log method on val

and TypeScript won't complain

because any does allow anything.

You are basically back to vanilla JavaScript with that.

With unknown, that's different

```
1 function process(val: unknown) {  
2   val.log();  
3 }
```

Now I'm getting an error

And what unknown does is it forces you, the developer,

to add some extra if checks

before you use the received value in a certain place.

```
1 function process(val: unknown) {
2     if (
3         typeof val === 'object' &&
4         !!val &&
5         'log' in val &&
6         typeof val.log === 'function'
7     ) {
8         val.log();
9     }
10 }
11
```

we can safely call log in there.

Optional Values & TypeScript

```
function generateError(msg?: string) {
    throw new Error(msg);
}

generateError();

type User = {
    name: string;
    age: number;
    role?: 'admin' | 'guest'
};
```

Nullish Coalescing (||) operator

```
> let input = '';
const didProvideInput = input ?? false;
console.log(didProvideInput);
```

```
> let input = '';  
const didProvideInput = input || false;  
console.log(didProvideInput);
```

false